

Automated Invoice Capture

Technology stack - reads supplier invoices, verifies, feeds Marathon

Runtime flow

```
Browser (React / Vite) --fetch /api--> FastAPI + Uvicorn
FastAPI -> pypdfium2 (PDF->PNG) -> Claude Opus 4.8 vision [or mock]
      -> python-stdnum (VAT/IBAN/Luhn) -> filesystem store
```

Backend - Python 3.14

Web framework	FastAPI + Uvicorn (ASGI)
File upload	python-multipart
PDF -> images	pypdfium2 (pip-only, no system deps)
Image handling	Pillow
AI extraction	anthropic SDK -> messages.parse (structured output)
Schema / validation	Pydantic v2 (the schema is the model contract)
Country validation	python-stdnum (VAT, IBAN checksum, Luhn check digit)
Money / dedup	stdlib decimal (exact), hashlib SHA-256 (duplicate fingerprint)
Storage	filesystem + in-memory (originals, page PNGs, result.json)

Frontend - JavaScript

UI library	React 18
Build / dev server	Vite 6 (proxies /api -> :8000)
Invoice rendering	server PNG + SVG overlay, coords normalized 0-1
Styling / HTTP	plain CSS / native fetch

AI / model layer

- Claude Opus 4.8 (claude-opus-4-8) vision - high-res boxes map 1:1 to page pixels
- Auth: ANTHROPIC_API_KEY / AUTH_TOKEN / ant CLI OAuth profile (SDK-resolved)
- Deterministic mock fallback - the whole app runs with no key/credit

Target stack - proposed (Section 4), not yet built

Database	PostgreSQL 16 (typed cols + JSONB)	today: in-memory + JSON
Object storage	MinIO / S3	today: local filesystem
Invoice render (FE)	PDF.js overlay	today: PNG + SVG
Text / OCR path	pdfplumber / Poppler, OCRmyPDF/Tesseract	today: vision only
Tax registration	VIES online lookup	today: structural only
Handover	transactional outbox -> Marathon	today: verify stores values
Packaging	Docker Compose / Kubernetes	today: run directly